LI.FI

FeeForwarder (v2.1.0)

Independent Security Review

SECURITY RESEARCHER

Sujith Somraaj (somraajsujith@gmail.com)

REPORT PREPARED BY

Sujith Somraaj

Table of Contents

1 About Researcher **1**

2 Disclaimer **1**

3 Scope **1**

4 Risk classification **1**
- 4.1 Impact . . . 1
- 4.2 Likelihood . . . 2
- 4.3 Action required for severity levels . . . 2

5 Executive Summary **2**

6 Findings **3**
- 6.1 Medium Risk . . . 3
  - 6.1.1 Whole-bps precision cannot represent fractional fee shares . . . 3
- 6.2 Low Risk . . . 4
  - 6.2.1 Fee-recipient callbacks can re-enter `GenericSwapFacetV3` and take the user's positive slippage . . . 4
  - 6.2.2 GenericSwapFacetV3-only usage is not enforced on-chain . . . 5
- 6.3 Informational . . . 5
  - 6.3.1 Additional edge-case tests would clarify fee semantics . . . 5
  - 6.3.2 Zero-value fee entries are emitted without a token transfer . . . 6
  - 6.3.3 Pre-existing Diamond balances are included in the fee base . . . 6
  - 6.3.4 Share multiplication can overflow for extreme token balances . . . 6

1 About Researcher

Sujith Somraaj is a distinguished security researcher and protocol engineer with over eight years of comprehensive experience in the web3 ecosystem.

In addition to working as a Lead Security Researcher at Spearbit, Sujith is also the security researcher and advisor for leading bridge protocol LI.FI and also is a former founding engineer and current security advisor at Superform, a yield aggregator with over $170M in TVL.

Sujith has experience working with protocols / funds including Coinbase, Solana Foundation, Layerzero, Edge Capital, Valinor, Kuru, MNX, Berachain, Optimism, Ondo, Sonic, Monad, Blast, ZkSync, Decent, Drips, SuperSushi Samurai, DistrictOne, Omni-X, Centrifuge, Superform-V2, Tea.xyz, Paintswap, Bitcorn, Sweep n' Flip, Byzantine Finance, Variational Finance, Satsbridge, Rova, Horizen, Earthfast, Angles and multiple other protocols. He has uncovered live bugs in Ethereum, Ripple, Uniswap, Morpho, Injective, Phantom, Cosmos and received bug-bounties from those projects.

Learn more about Sujith on sujithsomraaj.xyz or on cantina.xyz

2 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of that given smart contract(s) or blockchain software. i.e., the evaluation result does not guarantee against a hack (or) the non existence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, I always recommend proceeding with several audits and a public bug bounty program to ensure the security of smart contract(s). Lastly, the security audit is not an investment advice.

This review is done independently by the reviewer and is not entitled to any of the security agencies the researcher worked / may work with.

3 Scope

FeeForwarder	`src/Periphery/FeeForwarder.sol`

4 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

4.1 Impact

Rating	Definition
High	leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
Medium	global losses <10% or losses to only a subset of users, but still unacceptable.
Low	losses will be annoying but bearable — applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

4.2 Likelihood

Rating	Definition
High	almost certain to happen, easy to perform, or not easy but highly incentivized
Medium	only conditionally possible or incentivized, but still relatively likely
Low	requires stars to align, or little-to-no incentive

4.3 Action required for severity levels

Severity	Required action
Critical	Must fix as soon as possible (if already deployed)
High	Must fix (before deployment if not already deployed)
Medium	Should fix
Low	Could fix

5 Executive Summary

Over the course of 3 hours in total, LI.FI engaged with the researcher to audit the contracts described in section 3 of this document ("scope").

This is a differential review, and hence the scope is limited only to the changes introduced in the audit commit.

In this period of time a total of **7 issues** were found.

Project Summary

Project Name	**LIFI Contracts**
Project Type	Solidity Smart Contracts
Repository	lifinance/contracts
Commit	b1d029d25
Audit Timeline	October 29, 2026 - October 30, 2026
Methods	Manual Review
Documentation	High
Test Coverage	High

Issues Found

Critical Risk	**0**
High Risk	**0**
Medium Risk	**1**
Low Risk	**2**
Gas Optimizations	**0**
Informational	**4**
Total Issues	**7**

6 Findings

6.1 Medium Risk

6.1.1 Whole-bps precision cannot represent fractional fee shares

CONTEXT: FeeForwarder.sol#L27-L37

DESCRIPTION: `FeeBpsDistribution.bps` can express only whole basis points because it is divided by `10_000`. That is not enough precision for several fee splits the backend needs to send.

For example, a 5% LI.FI share of a 30 bps fee is 1.5 bps, while a 1% share is 0.3 bps. Neither value can be encoded. Rounding 0.3 bps down removes that recipient's fee entirely; rounding it up to 1 bp charges more than three times the intended amount. The same problem affects integrator fees such as 12.5 bps and shares derived from already-rounded token amounts.

This is not token dust rounding. The percentage itself is rounded before the contract sees it, so increasing the swap amount does not improve the relative error. Falling back to `forwardERC20Fees` also brings back the original problem of computing an absolute fee from a minimum output rather than the realized output.

PROOF OF CONCEPT:

The following test can be added to `FeeForwarderTest`. It uses 100 USDC, where the intended 1.5 bps fee is 15,000 base units. The two closest values accepted by the contract produce 10,000 and 20,000 base units instead.

```
function test_ForwardERC20FeesBpsCannotRepresentOnePointFiveBps()
    public
{
    uint256 balance = 100 * 10 ** usdc.decimals();
    uint256 expectedFee = 15_000; // 1.5 bps of 100 USDC

    FeeForwarder.FeeBpsDistribution[]
        memory distributions = new FeeForwarder.FeeBpsDistribution[](1);
    distributions[0] = FeeForwarder.FeeBpsDistribution({
        recipient: USER_RECEIVER,
        bps: 1
    });

    deal(address(usdc), USER_SENDER, balance);
    deal(address(usdc), USER_RECEIVER, 0);

    vm.startPrank(USER_SENDER);

    usdc.approve(address(feeForwarder), type(uint256).max);
    feeForwarder.forwardERC20FeesBps(address(usdc), distributions);

    vm.stopPrank();

    uint256 oneBpsFee = usdc.balanceOf(USER_RECEIVER);

    deal(address(usdc), USER_SENDER, balance);
    deal(address(usdc), USER_REFUND, 0);
    distributions[0] = FeeForwarder.FeeBpsDistribution({
        recipient: USER_REFUND,
        bps: 2
    });

    vm.prank(USER_SENDER);
    feeForwarder.forwardERC20FeesBps(address(usdc), distributions);

    uint256 twoBpsFee = usdc.balanceOf(USER_REFUND);

    assertEq(oneBpsFee, 10_000);
    assertEq(twoBpsFee, 20_000);
    assertLt(oneBpsFee, expectedFee);
    assertGt(twoBpsFee, expectedFee);
}
```

RECOMMENDATION: Represent each share with a finer fixed-point unit. One option is a `uint64 share` where `1e18` means 100%, then calculate each amount with `FixedPointMathLib.fullMulDiv(balance, share, 1e18)` and reject a cumulative share above `1e18`.

Because changing the tuple member changes the function selector, update the function and error names, build attestation, documentation, backend encoder, whitelist configuration, and rollout data together. Add regression cases for the equivalents of 0.3, 1.5, and 12.5 bps.

LI.FI: Fixed in 6ab2ec0f.

RESEARCHER: Verified fix.

6.2 Low Risk

6.2.1 Fee-recipient callbacks can re-enter `GenericSwapFacetV3` and take the user's positive slippage

CONTEXT: FeeForwarder.sol#L86-L91, FeeForwarder.sol#L170-L173, GenericSwapFacetV3.sol#L442-L444, GenericSwapFacetV3.sol#L517-L524, FeeForwarder.md#L147-L148

DESCRIPTION: Every FeeForwarder entrypoint hands control to each recipient: `forwardNativeFees` sends native value with all gas forwarded, and the two ERC20 functions call `transferFrom`, which on a hook-bearing token (ERC777, ERC1363 and similar) notifies the recipient. Within `FeeForwarder` itself that callback is harmless, and the existing `ReentrantFeeRecipient` tests prove it: a nested `forwardNativeFees` cannot spend the outer call's `msg.value`.

The callback is not harmless when the caller is `GenericSwapFacetV3`, which is the intended caller. The facet has no reentrancy guard, funds native swap steps from its own balance rather than from `msg.value`, keeps the unlimited allowance it granted the forwarder, and pays the receiver the Diamond's **full** balance of the output asset. A recipient that re-enters the Diamond from its callback therefore runs a second, nested route against the outer route's in-flight funds. The nested route only has to reference allowlisted steps, so it can be an ordinary `forwardNativeFees` / `forwardERC20Fees` step paying the attacker, and it passes its own `minAmountOut` of zero. The outer route then still succeeds as long as the attacker leaves at least `minAmountOut` behind for the receiver.

Reproduced (`FeeForwarderReentrancyPoC.t.sol`, both tests pass against this branch unchanged):

Scenario	Fee allocated	Taken via nested route	Receiver gets
ERC20 hook token, `forwardERC20FeesBps` 10%	100	200	700 (its `minAmountOut`)
Native, `forwardNativeFees` 100	100	200	700 (its `minAmountOut`)

The attacker is a fee recipient, which is an integrator-registered address, so the trust assumption is "integrator", not "anyone". The amount at risk per route is bounded by the user's slippage allowance (output minus `minAmountOut`) plus any stray Diamond balance of that asset, and only routes whose fee step pays a contract recipient in native or in a hook token are exposed. An integrator that already receives a fee gains the ability to exceed its agreed fee silently on every such route.

RECOMMENDATION: Fix at the composed caller, not in FeeForwarder. A route-wide `nonReentrant` on every `GenericSwapFacetV3` entrypoint closes this for all callback sources (fee recipients, hook tokens, DEX callbacks), and belongs in the facet's next version bump alongside the EXSC-850 items that already target the same balance-funded `fromAmount` behaviour. A `nonReentrant` on FeeForwarder alone would block the demonstrated recursion but not a callback that re-enters through a different allowlisted step, so treat it as optional defense-in-depth only.

For this PR, correct the two documentation bullets: state that a fee recipient's callback can re-enter the Diamond while the route's output is still in flight, that `GenericSwapFacetV3` currently does not prevent this, and that recipients are therefore trusted for funds up to the route's slippage allowance, not only for liveness. Fee recipients that are contracts should be limited to known integrator addresses until the facet guard ships.

LI.FI: Acknowledged - the re-entry path is in GenericSwapFacetV3 and predates v2.1.0. The route-wide nonReentrant will come with the facet's next version bump. In this PR, the docs

now state that recipients can re-enter the Diamond and are trusted for funds up to the slippage allowance, and that contract recipients must be known integrators until the guard ships: https://github.com/lifinance/contracts/commit/17c4bdbc6a10ffb27b2b8a5928f97ae8e7189caf

RESEARCHER: Acknowledged.

6.2.2 GenericSwapFacetV3-only usage is not enforced on-chain

CONTEXT: FeeForwarder.sol#L97-L101, SwapperV2.sol#L190-L218

DESCRIPTION: `forwardERC20FeesBps` is documented as a `GenericSwapFacetV3`-only step, but that restriction is not enforced on-chain. The Diamond stores one contract-and-selector allowlist, and both `GenericSwapFacetV3` and facets using `SwapperV2` consult the same entry. Once the new FeeForwarder selector is allowed for generic swaps, a `SwapperV2` bridge route can call it as well.

This matters because `SwapperV2` credits only the increase over the Diamond's initial balance. `forwardERC20FeesBps`, on the other hand, charges against the full balance. A token donation therefore increases the fee without increasing the amount credited to the bridge. Depending on the donated balance and the route's available slippage, the bridge receives less than intended or the route reverts.

The attacker has to donate the tokens, and the route builder must first violate the documented integration rule, so the issue is limited. The missing piece is that the current documentation does not say the restriction relies entirely on off-chain route construction.

RECOMMENDATION: Reject this selector in the route builder for every `SwapperV2` bridge route. Also add a sentence to the Security Considerations section stating that `forwardERC20FeesBps` is not restricted to `GenericSwapFacetV3` on-chain and that route builders must prevent its use in bridge routes.

An on-chain restriction would require execution-context-aware permissions or a separate callable surface; the current global allowlist cannot distinguish which facet initiated the step.

LI.FI: Fixed in 13e49937.

RESEARCHER: Verified fix.

6.3 Informational

6.3.1 Additional edge-case tests would clarify fee semantics

CONTEXT: FeeForwarder.t.sol#L785-L1097, GenericSwapFacetV3.t.sol#L1499-L1637

DESCRIPTION: The new tests cover the main calculation, rounding, zero shares, invalid recipients, the 10,000 bps limit, missing allowance, a pre-existing Diamond balance, and the net `minAmountOut` requirement. Three useful edge cases remain uncovered:

- A fee-on-transfer token, to make the distinction between the computed event amount, the caller's balance reduction, and the recipient's actual receipt explicit.
- The fee step's `AssetSwapped` event, especially its quoted `fromAmount` and post-fee full-balance `toAmount`.
- A three-entry array whose cumulative total first exceeds 10,000 at the last index, confirming the reported running total and atomic rollback of the earlier transfers.

These are coverage improvements rather than evidence of an implementation defect.

RECOMMENDATION: Add focused tests for the three cases above. If fee-on-transfer output tokens are intentionally unsupported, encode that decision in a test and state it in the documentation instead of leaving the behavior implicit.

LI.FI: Fixed in ba8f295.

RESEARCHER: Verified fix.

6.3.2 Zero-value fee entries are emitted without a token transfer

CONTEXT: FeeForwarder.sol#L129-L142

DESCRIPTION: `forwardERC20Fees` and `forwardERC20FeesBps` deliberately share the `FeesForwarded` event. The bps function converts its input into the existing `FeeDistribution` shape and emits the computed token amounts, which keeps current backend parsing compatible.

An entry whose amount rounds to zero remains in the event with `amount == 0`, even though the contract skips the corresponding `transferFrom` call. Consumers should therefore treat the event as the result of the fee calculation, not as proof that one token transfer occurred for every array entry.

This behavior is documented and tested.

RECOMMENDATION: No code change is required. Keep the event semantics in the documentation, and ensure indexers do not infer an ERC20 transfer from a zero-valued distribution entry.

LI.FI: Acknowledged - zero shares skip the transfer because some tokens revert on zero-value transfers, but they stay in FeesForwarded so the event keeps one entry per recipient, in input order.

RESEARCHER: Acknowledged.

6.3.3 Pre-existing Diamond balances are included in the fee base

CONTEXT: FeeForwarder.sol#L104-L129

DESCRIPTION: `forwardERC20FeesBps` reads `balanceOf(msg.sender)` once and uses that full value as the fee base. When the caller is the Diamond, the base includes both the current swap output and any tokens the Diamond already held.

This behavior is intentional and internally consistent for `GenericSwapFacetV3`: after the fee step, the facet sends its full remaining output-token balance to the receiver. A pre-existing balance therefore increases both the fee and the receiver's payout. The behavior is documented and covered by an integration test.

RECOMMENDATION: No code change is required. Keep the full-balance behavior explicit in the documentation and retain the pre-existing-balance regression test so a later refactor does not silently change the accounting basis.

LI.FI: Acknowledged - intentional. GenericSwapFacetV3 pays its full remaining output-token balance to the receiver, so a pre-existing balance raises the fee and the payout alike. This is documented, and the pre-existing-balance integration test stays as a regression guard.

RESEARCHER: Acknowledged.

6.3.4 Share multiplication can overflow for extreme token balances

CONTEXT: FeeForwarder.sol#L120-L130

DESCRIPTION: The share calculation multiplies the caller's balance by `bps` before dividing by `10_000`. Solidity's checked arithmetic makes this safe, but a token reporting a sufficiently large balance causes the multiplication to revert before any transfer is attempted.

At 10,000 bps, the threshold is approximately `type(uint256).max / 10_000`, or `1.16e73` base units. No realistic token balance approaches that value. A malicious or deliberately extreme token can only make its own route revert, so this is a liveness observation rather than a funds-at-risk issue.

RECOMMENDATION: No immediate change is required. If the calculation is touched while increasing share precision, use `FixedPointMathLib.fullMulDiv` so the full `uint256` input domain remains valid. Do not place the multiplication in an `unchecked` block.

LI.FI: Fixed by solving the medium issue in 6ab2ec0fb.

RESEARCHER: Verified fix.